CS 5402: Introduction to Data Mining

SP2026

(Lecture -7)

Decision Tree

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

MISSOURI S&T

Outline

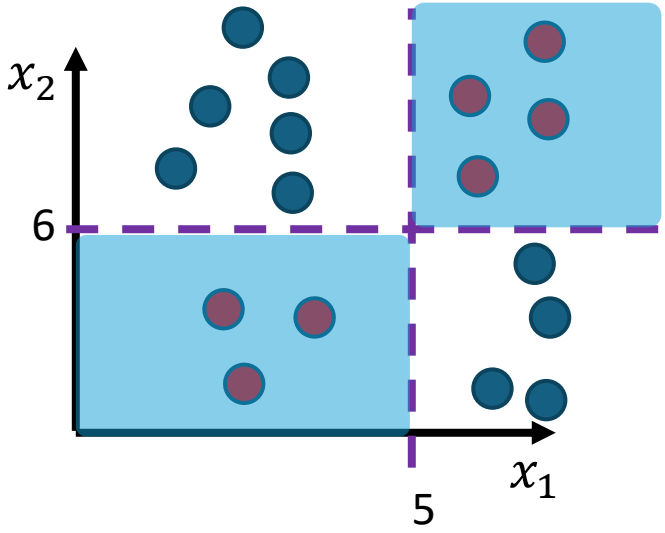
- Decision Tree Classifier
- Feature Selection
- Build a Decision Tree Classifier
- Pruning

Rule-Based Classifier

- Classify records by using a collection of "if…then…" rules

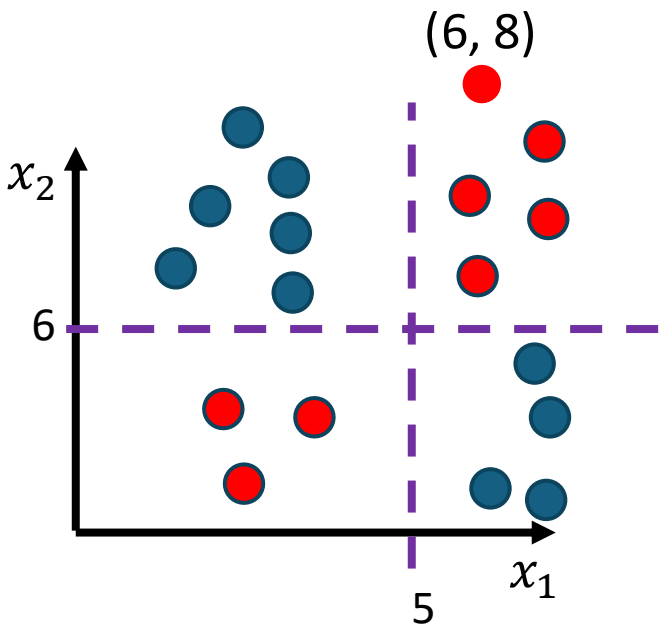
- Rule: (*Condition*) $\rightarrow y$
  - where
    - *Condition* is a conjunction of tests on attributes
    - y is the class label
  - Examples of classification rules:
    - (Blood Type=Warm) $\wedge$ (Lay Eggs=Yes) $\rightarrow$ Birds
    - (Taxable Income < 50K) $\wedge$ (Refund=Yes) $\rightarrow$ Evade=No

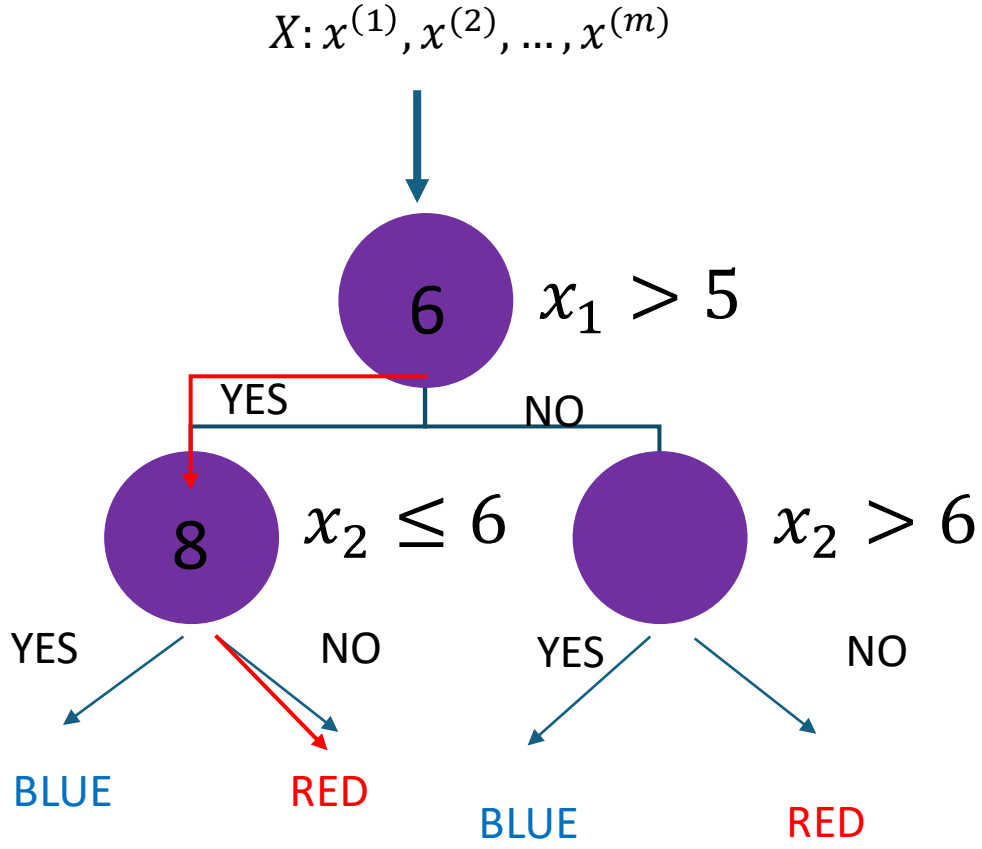
Rule-Based Classifier Example



If $x_1 > 5$ and $x_2 > 6$, then predict RED
If $x_1 < 5$ and $x_2 < 6$, then predict RED
… and so on

This is Decision Tree!





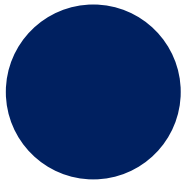
What is a decision tree?

A decision tree:

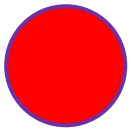
- Recursively splits data based on feature values, at each node:
  I. Consider all candidate features
  II. Evaluate “how good” each split is
  III. Choose the **best split**
  IV. Recurse on subsets

- Each internal node: a feature test
- Each leaf node: a predicted class

Decision Tree: Terminology



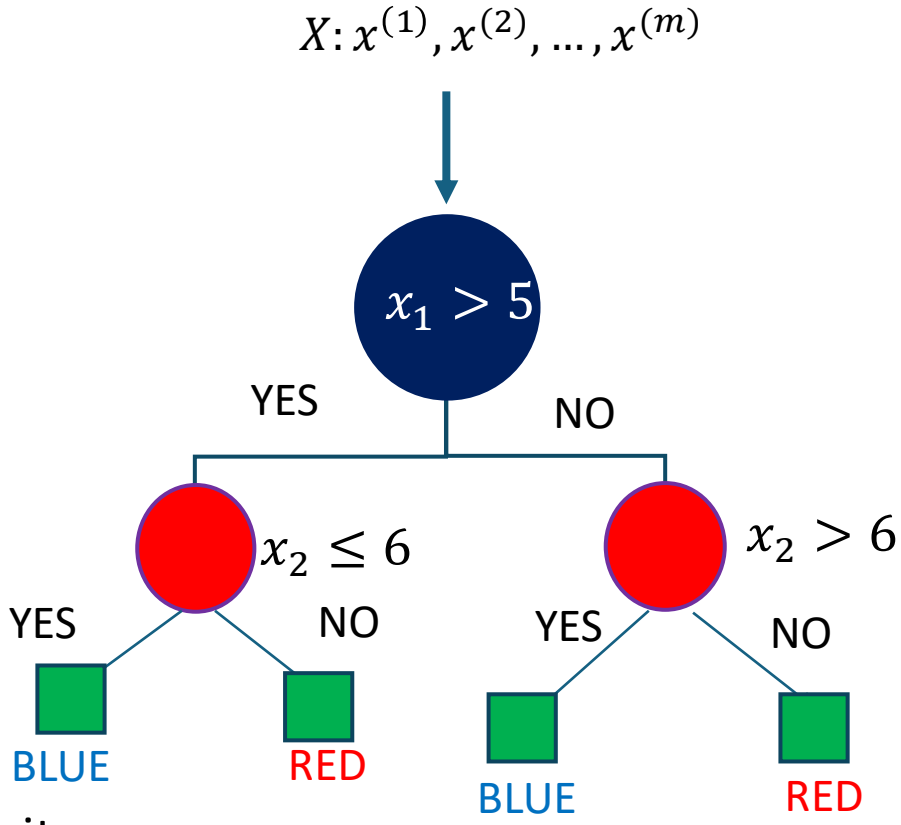
Root node. Has only nodes below it.



Branch node. Has nodes both above and below it.



Leaf node. Has only nodes above it.



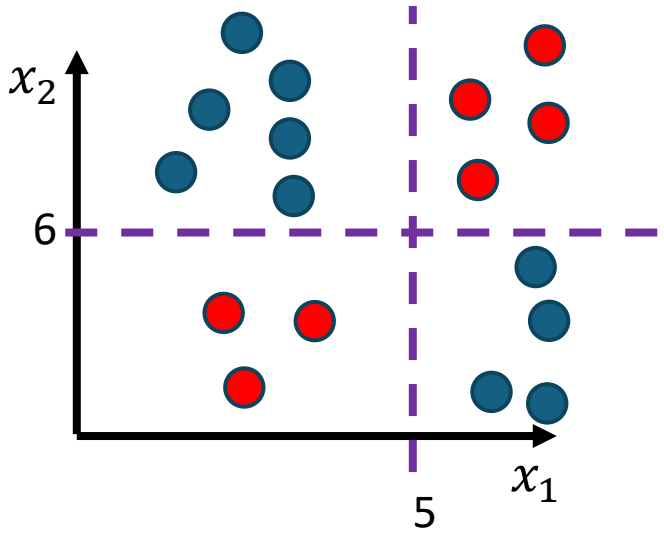
How to construct a decision tree?

Two questions:

Q1: Which feature to split on?

Q2: What value of the feature to split on?

Name	Give Birth	Lay Eggs	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	no	yes	mammals
python	no	yes	no	no	no	reptiles
salmon	no	yes	no	yes	no	fishes
whale	yes	no	no	yes	no	mammals
frog	no	yes	no	sometimes	yes	amphibians
komodo	no	yes	no	no	yes	reptiles
bat	yes	no	yes	no	yes	mammals
pigeon	no	yes	yes	no	yes	birds
cat	yes	no	no	no	yes	mammals
leopard shark	yes	no	no	yes	no	fishes
turtle	no	yes	no	sometimes	yes	reptiles
penguin	no	yes	no	sometimes	yes	birds
porcupine	yes	no	no	no	yes	mammals
eel	no	yes	no	yes	no	fishes
salamander	no	yes	no	sometimes	yes	amphibians
gila monster	no	yes	no	no	yes	reptiles
platypus	no	yes	no	no	yes	mammals
owl	no	yes	yes	no	yes	birds
dolphin	yes	no	no	yes	no	mammals
eagle	no	yes	yes	no	yes	birds



Outline (2)

- Decision Tree Classifier
- Feature Selection
- Build a Decision Tree Classifier
- Pruning

Feature Selection

Key Problem: choosing which attribute to split a given set of examples.

A good split:

- Makes child nodes more pure.
- Ideally, each node contains samples from only one class.

We need impurity measures:

- Entropy -> Information Gain
- Gini Index

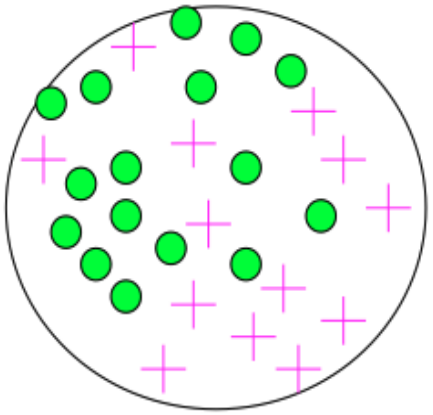
$$\text{Entropy} = -\sum_{i=1}^{k} p_i \cdot \log_2(p_i)$$

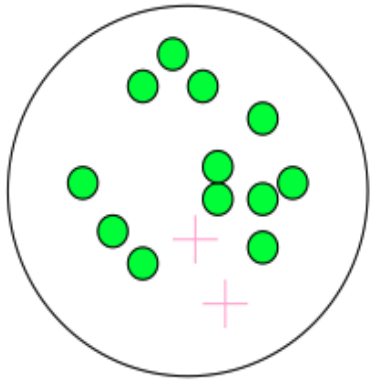
$$\text{Gini} = \sum_{i=1}^{k} p_i (1 - p_i) = 1 - \sum_{i=1}^{k} p_i^2$$

Information Gain: Entropy

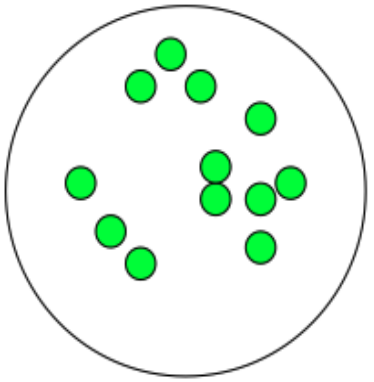
- Entropy
  - Measures the level of entropy in a group of examples.

High entropy





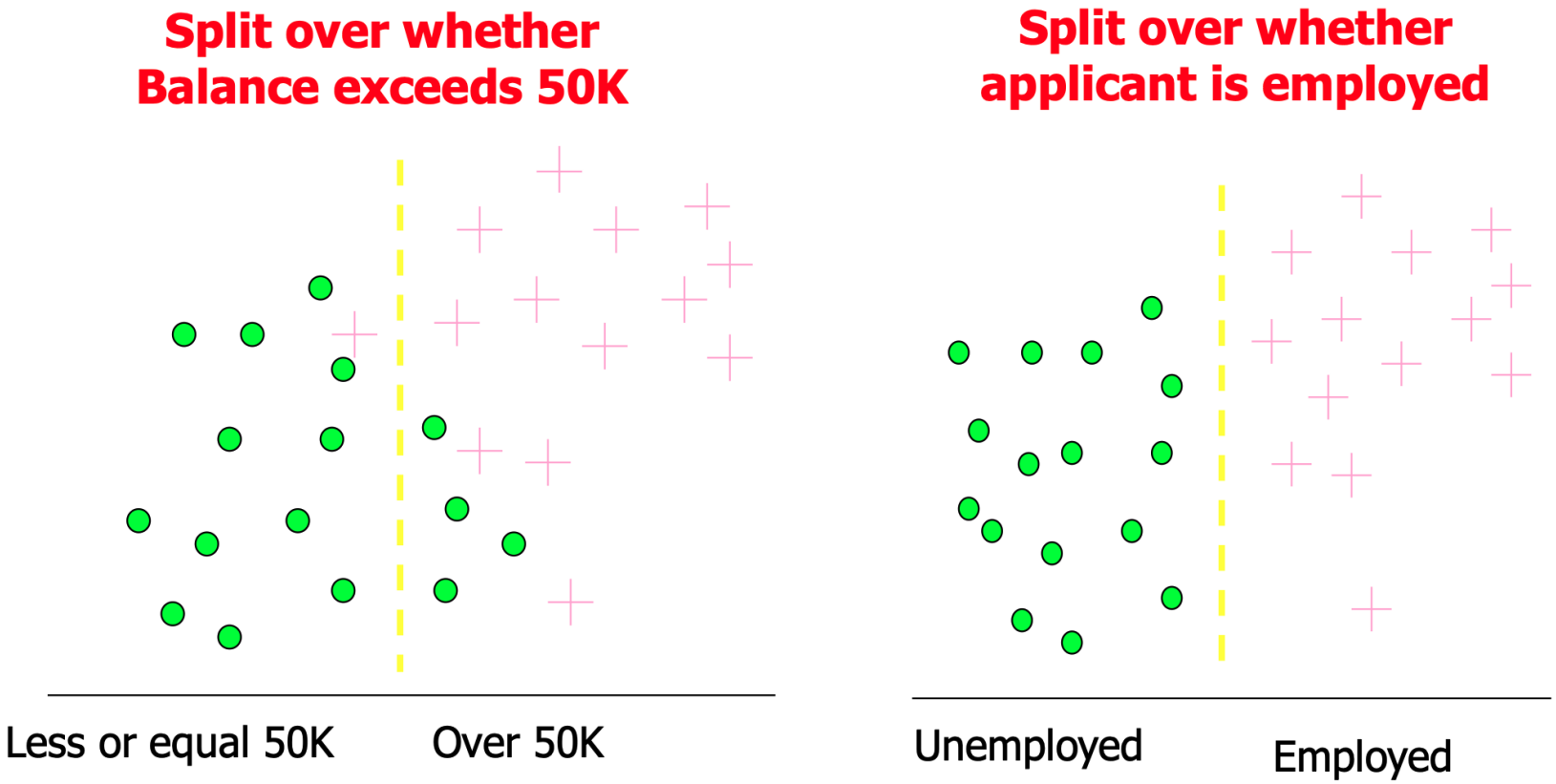
Minimum entropy



Based on slide by Eric Eaton

Information Gain: from entropy to information

- Which test is more informative?



Based on slide by Eric Eaton

Information Gain Definition

- Information Gain is the *mutual information* between input attribute A and target variable Y. It tells us how important a given attribute of the feature vectors is.
- We will use it to decide the ordering of attributes in the nodes of a decision tree.
- Information Gain is the expected reduction in entropy of target variable Y for data sample D, due to sorting on variable A.

$$G(D, A) = I(Y, A) = H(Y) - H(Y|A)$$

Cont'd

$$G(D,A) = I(Y,A) = H(Y) - H(Y|A)$$

$$H(\mathrm{Y}) = -\sum_{\mathrm{y}\in\mathrm{Y}} p(\mathrm{y}) \log_2 p(\mathrm{y})$$

$$H(\mathrm{Y}|\mathrm{A}) = -\sum_{\mathrm{y}\in\mathrm{Y}} \sum_{\mathrm{a}\in\mathrm{A}} p(\mathrm{a}, y) \log_2 p(\mathrm{y}|\mathrm{a})$$

$$H(Y \mid A) = -\sum_{y\in Y} \sum_{a\in\{a_1,a_2\}} p(y,a) \log_2 p(y \mid a)$$

$$= -\sum_{y\in Y} [p(y,a_1) \log_2 p(y \mid a_1) + p(y,a_2) \cdot \log_2 p(y \mid a_2)]$$

$$p(y \mid a_1) \cdot p(a_1) \qquad p(y \mid a_2) \cdot p(a_2)$$

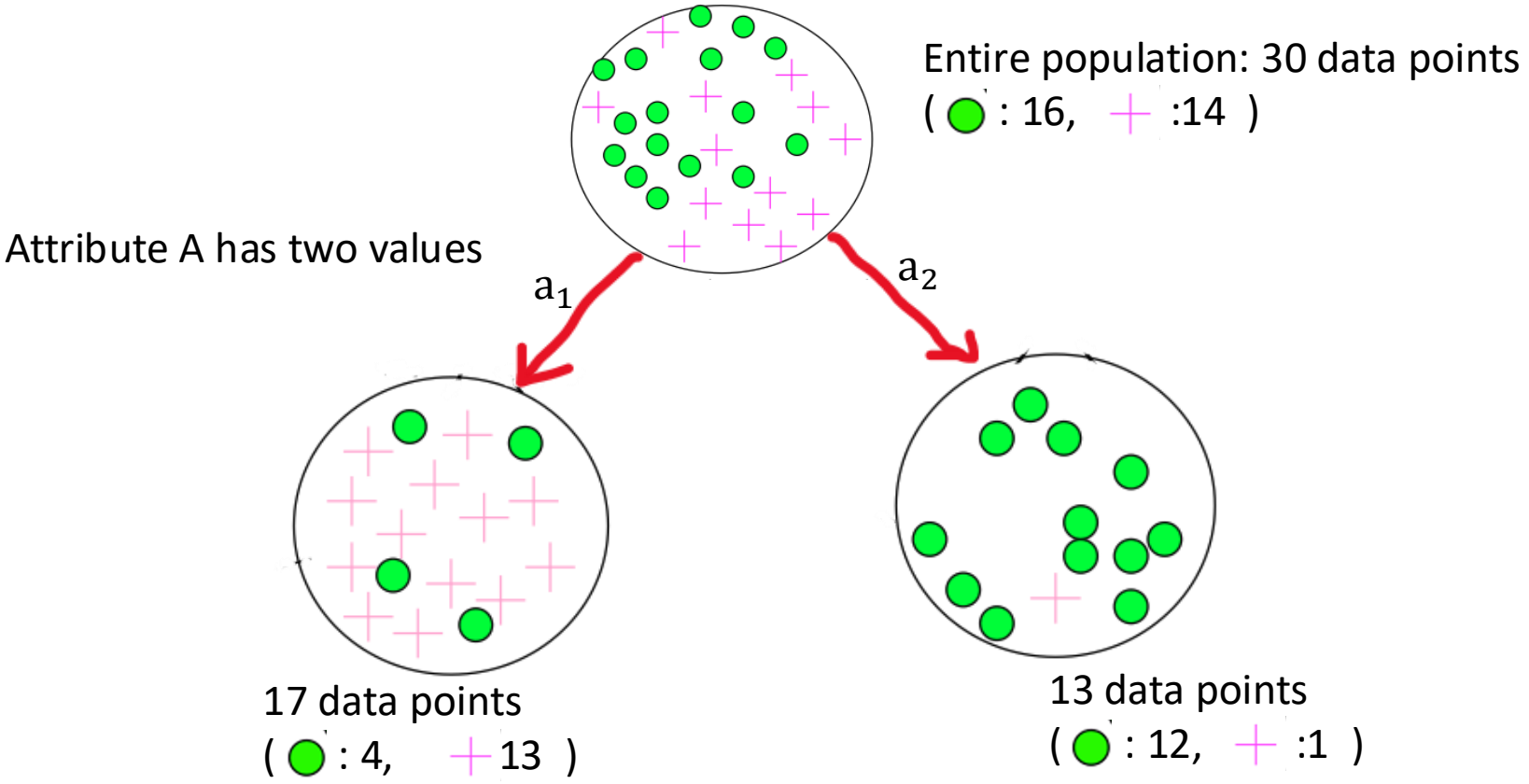
$$= -P(a_1) \sum_{y\in Y} p(y \mid a_1) \cdot \log_2 p(y \mid a_1)$$

$$-P(a_2) \sum_{y\in Y} p(y \mid a_2) \cdot \log 2p(y \mid a_2)$$

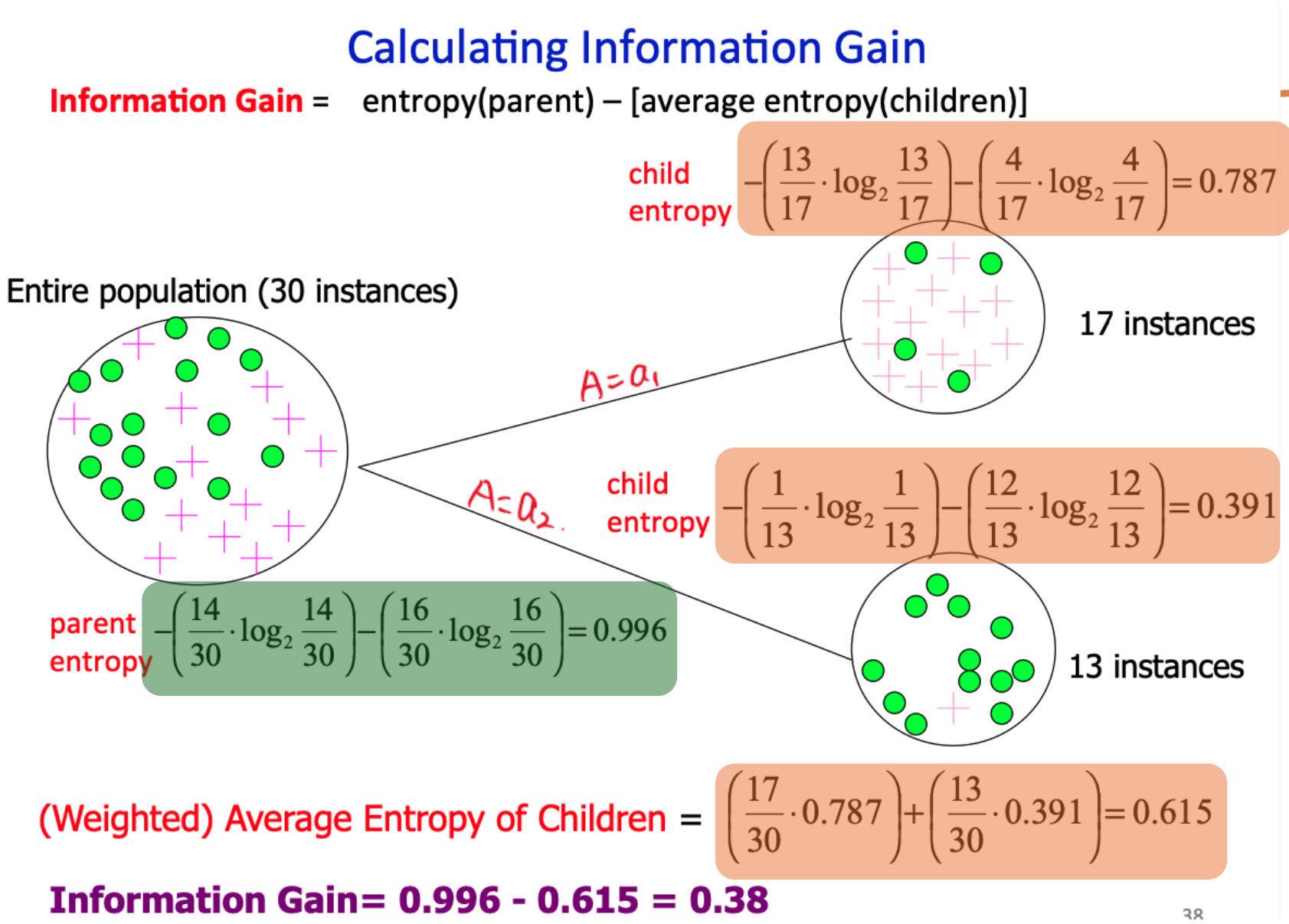
Entropy in subgroup

Name	Give Birth	Lay Eggs	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	no	yes	mammals
python	no	yes	no	no	no	reptiles
salmon	no	yes	no	yes	no	fishes
whale	yes	no	no	yes	no	mammals
frog	no	yes	no	sometimes	yes	amphibians
komodo	no	yes	no	no	yes	reptiles
bat	yes	no	yes	no	yes	mammals
pigeon	no	yes	yes	no	yes	birds
cat	yes	no	no	no	yes	mammals
leopard shark	yes	no	no	yes	no	fishes
turtle	no	yes	no	sometimes	yes	reptiles
penguin	no	yes	no	sometimes	yes	birds
porcupine	yes	no	no	no	yes	mammals
eel	no	yes	no	yes	no	fishes
salamander	no	yes	no	sometimes	yes	amphibians
gila monster	no	yes	no	no	yes	reptiles
platypus	no	yes	no	no	yes	mammals
owl	no	yes	yes	no	yes	birds
dolphin	yes	no	no	yes	no	mammals
eagle	no	yes	yes	no	yes	birds

Example: Information Gain



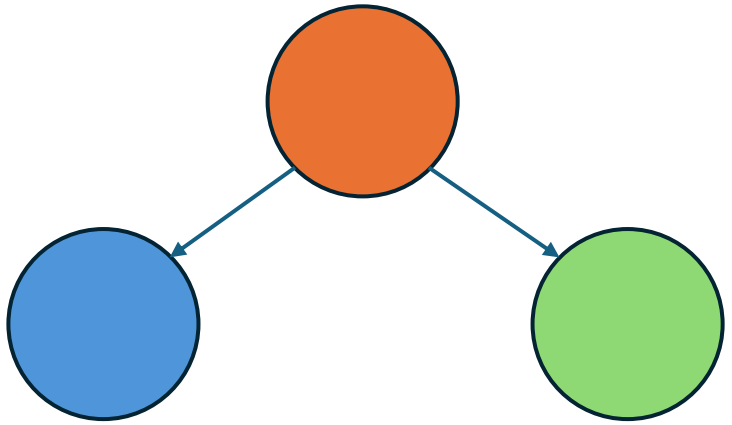
Example: Information Gain (Cont'd)



Question-1: which attribute has higher information gain?

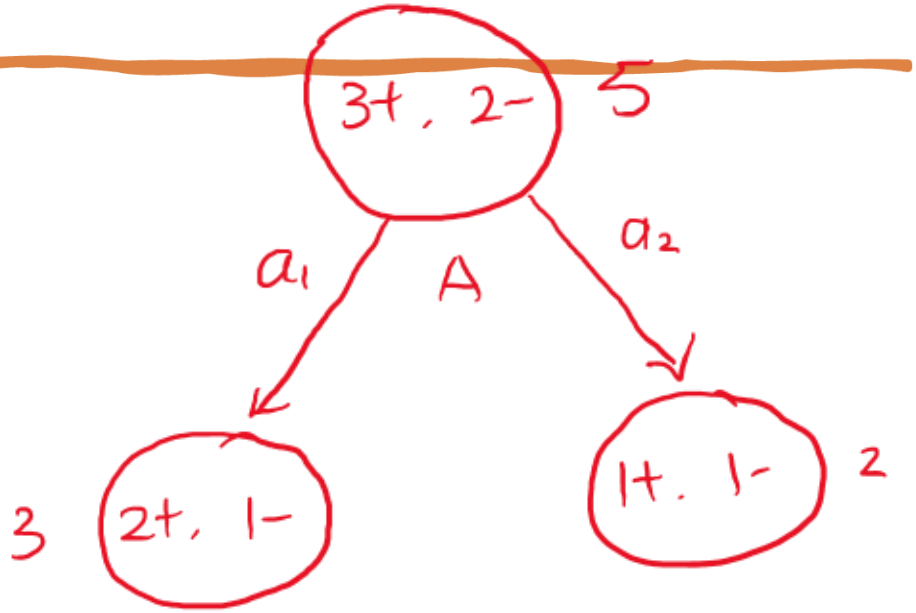
A	B	Yes/No
a_1	b_1	Yes
a_2	b_2	No
a_1	b_1	Yes
a_2	b_1	Yes
a_1	b_2	No

$G(D, A)$? $G(D, \mathrm{B})$



Question-1: $G(D, A) = ?$

A	B	Yes/No
a_1	b_1	Yes
a_2	b_2	No
a_1	b_1	Yes
a_2	b_1	Yes
a_1	b_2	No



$$H(Y) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.9710$$

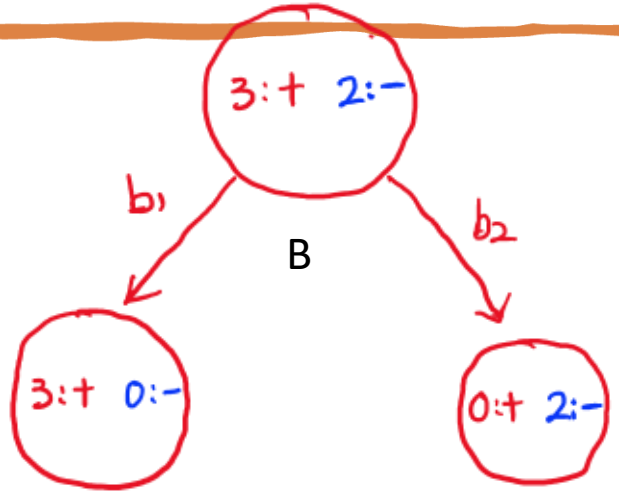
$$H(Y \mid a_1) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$H(Y \mid a_2) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$$

$$\begin{aligned} G(D, A) &= I(Y, A) \\ &= H(Y) - P(a_1)H(Y \mid a_1) - P(a_2)H(Y \mid a_2) \\ &= 0.9710 - \frac{3}{5} \times 0.9183 - \frac{2}{5} \times 1.0 \\ &= 0.02 \end{aligned}$$

Question-1: $G(D, B) = ?$

A	B	Yes/No
a_1	b_1	Yes
a_2	b_2	No
a_1	b_1	Yes
a_2	b_1	Yes
a_1	b_2	No



$$H(Y) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.9710$$

$$H(Y \mid b_1) = 0$$

$$H(Y \mid b_2) = 0$$

$$\begin{aligned} G(D, B) &= I(Y, B) \\ &= H(Y) - H(Y \mid B) \\ &= H(Y) - P(b_1)H(Y \mid b_1) - P(b_2)H(Y \mid b_2) \\ &= 0.971 - \frac{3}{5} \times 0 - \frac{2}{5} \times 0 \\ &= 0.971 \end{aligned}$$

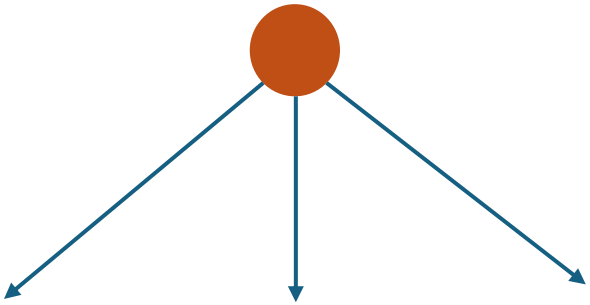
Outline (3)

- Decision Tree Classifier
- Feature Selection
- Build a Decision Tree Classifier
- Pruning

Using information Gain to construct a Decision Tree

Node 1:

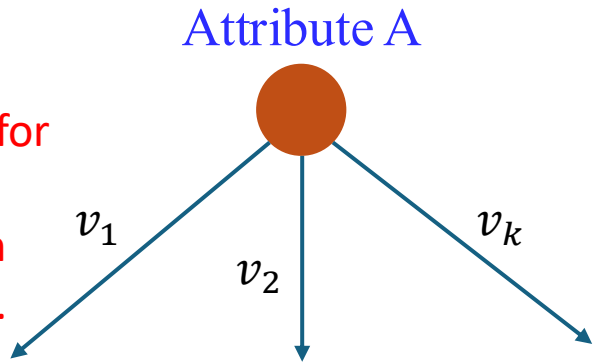
What feature should be used?



Name	Give Birth	Lay Eggs	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	no	yes	mammals
python	no	yes	no	no	no	reptiles
salmon	no	yes	no	yes	no	fishes
whale	yes	no	no	yes	no	mammals
frog	no	yes	no	sometimes	yes	amphibians
komodo	no	yes	no	no	yes	reptiles
bat	yes	no	yes	no	yes	mammals
pigeon	no	yes	yes	no	yes	birds
cat	yes	no	no	no	yes	mammals
leopard shark	yes	no	no	yes	no	fishes
turtle	no	yes	no	sometimes	yes	reptiles
penguin	no	yes	no	sometimes	yes	birds
porcupine	yes	no	no	no	yes	mammals
eel	no	yes	no	yes	no	fishes
salamander	no	yes	no	sometimes	yes	amphibians
gila monster	no	yes	no	no	yes	reptiles
platypus	no	yes	no	no	yes	mammals
owl	no	yes	yes	no	yes	birds
dolphin	yes	no	no	yes	no	mammals
eagle	no	yes	yes	no	yes	birds

Using information Gain to construct a Decision Tree (Cont'd)

Step-II: Construct child nodes for each value of A. Each has an associated subset of vectors in which A has a particular value.



Step-I: Choose the attribute A with highest information gain for the full training set at the root of the tree.

Set $X_1 = \{x \in X | value(A) = v_1\}$

Repeat recursively.

A	B	Yes/No
a_1	b_1	Yes
a_2	b_2	No
a_1	b_1	Yes
a_2	b_1	Yes
a_1	b_2	No

Example: Step-1

Calculate Information Gain for each feature

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Sunny	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	*No*
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*

G(D, Outlook) = 0.246

G(D, Temperature) = 0.029

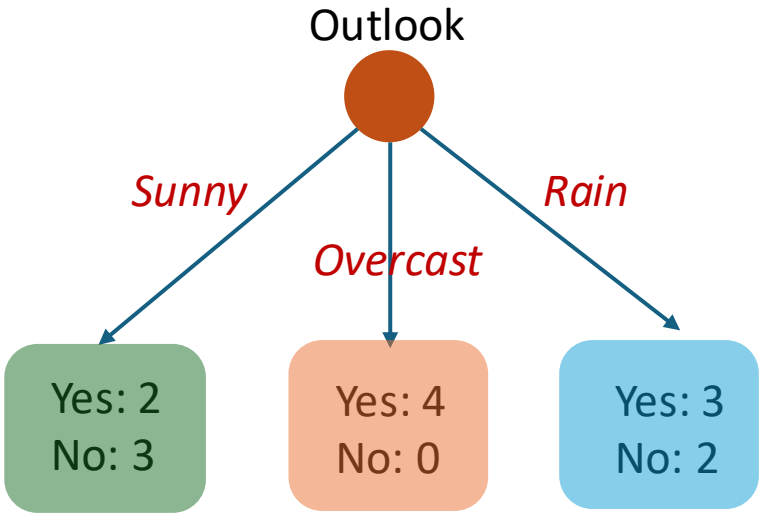
G(D, Humidity) = 0.151

G(D, Wind) = 0.048

Example: Step-2

Pick the one with the largest Information Gain as Root.

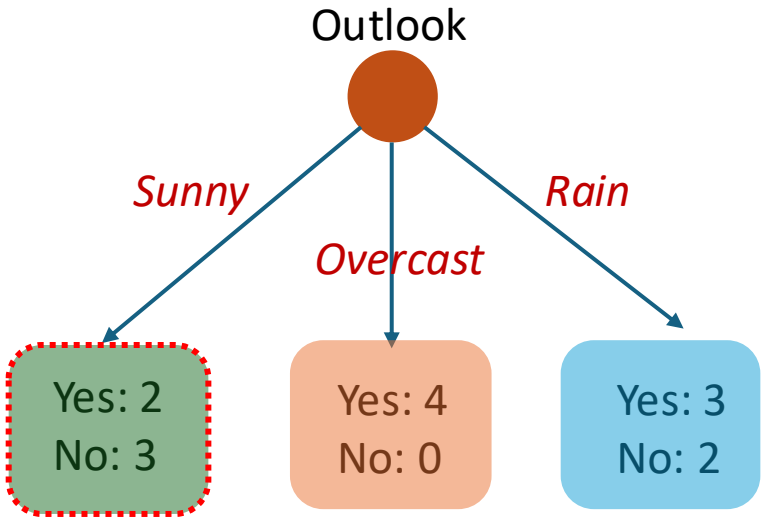
Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Sunny	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*



Example: Repeat Step-1

Calculate Information Gain for each feature in the subset

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot			
Day14	Rain	Mild			



Which attribute should be tested here?

$$Gain\ (S_{sunny}, Humidity) = .970 - (3/5)\ 0.0 - (2/5)\ 0.0 = .970$$

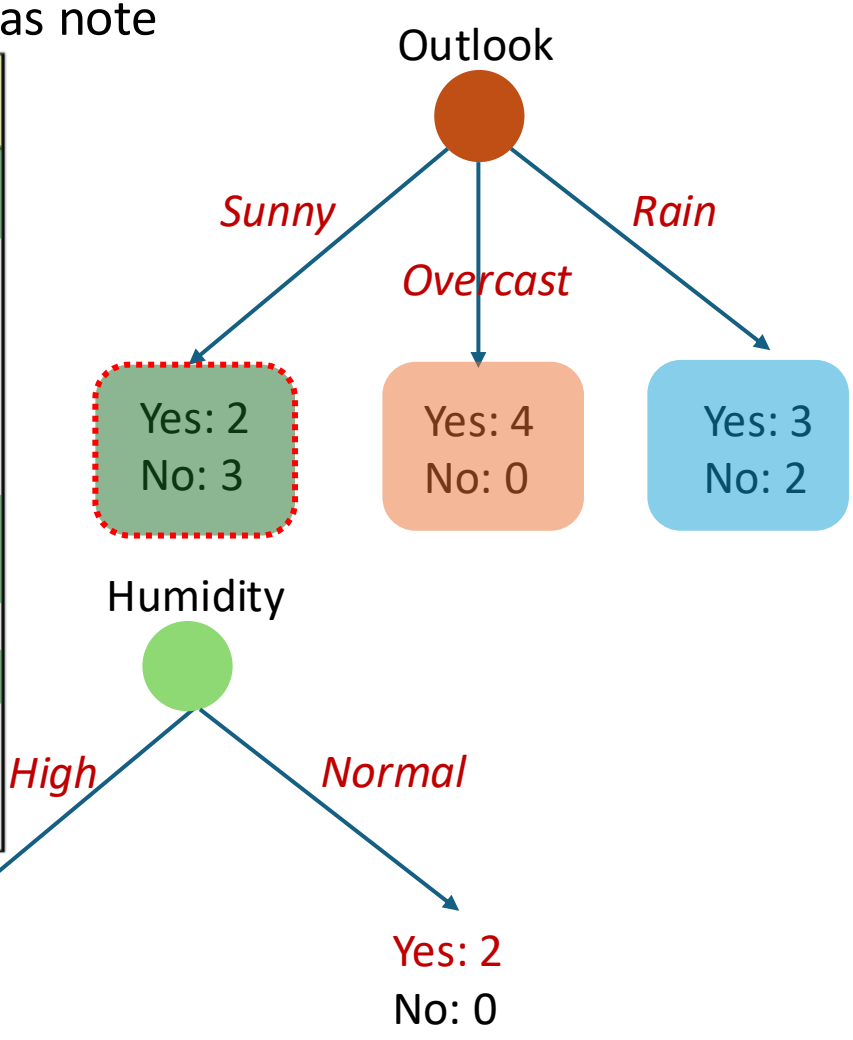
$$Gain\ (S_{sunny}, Temperature) = .970 - (2/5)\ 0.0 - (2/5)\ 1.0 - (1/5)\ 0.0 = .570$$

$$Gain\ (S_{sunny}, Wind) = .970 - (2/5)\ 1.0 - (3/5)\ .918 = .019$$

Example: Repeat Step-2

pick the one with highest information gain as note

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*



Next: The Problem with Information Gain

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Sunny	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	*No*
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*

The Flaw: Information Gain is inherently biased. It naturally favors attributes that have *many* unique values.

H("Day") = 3.81

G(D, "Day") = 0.97-0=0.97

Information Gain Ratio (IGR)

Disadvantage of information gain:

- It prefers attributes with large number of values that split the data into small, pure subsets

Information Gain Ratio:

- Normalizes the Information Gain by penalizing attributes that have too many splits.

$$G_R(D, A) = \frac{G(D, A)}{H(A)}$$

$$\frac{\text{G(D, “Day”)}}{\text{H(“Day”)}} = \frac{0.97}{3.81} = 0.255$$

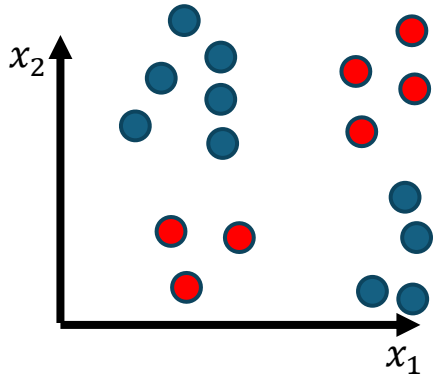
The problem with different attributes:

Categorical features VS. Continuous features

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Sunny	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	*No*
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*

ID	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Class Label
1	3.2	75.4	1.4	89.2	4.6	Yes
2	5.6	32.1	9.1	24.7	7.3	No
3	2.1	68.5	5.3	12.3	9.0	Yes
4	8.9	25.7	7.6	31.9	4.8	No
5	4.4	82.3	3.7	55.1	6.1	Yes
6	7.1	19.8	6.5	98.4	3.2	No
7	3.5	47.6	8.3	22.8	5.6	Yes
8	6.9	95.2	2.1	74.5	1.8	No
9	5.2	36.9	7.9	80.3	2.9	Yes
10	4.8	63.7	1.5	39.6	7.7	No

What value of feature to split on? (1)

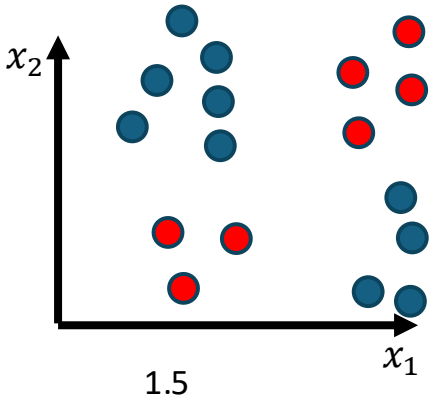


For each feature, do:

a. Sort the points from lowest to highest

x_1	Mid-points $x_{mid,1}$
1	
2	
2.4	
3.6	
6	

What value of feature to split on? (2)



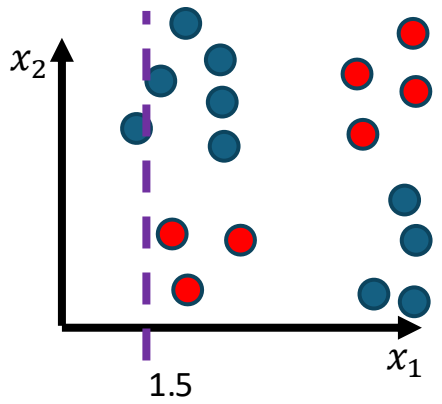
For each feature, do:

a. Sort the points from lowest to highest
b. Compute successive midpoints between each point in the dataset

x_1	Mid-points $x_{mid,1}$
1	1.5
2	2.2
2.4	3.0
3.6	4.8
6	…

$$x_{mid,1} = \frac{x_i + x_{i+1}}{2}$$

What value of feature to split on? (3)



For each feature, do:

a. Sort the points from lowest to highest
b. Compute successive midpoints between each point in the dataset

c. Compute Entropy or Gini measures for all the midpoints

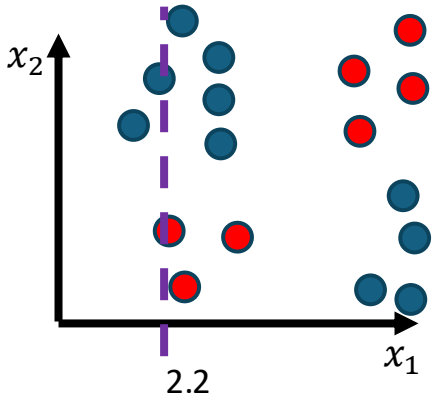
So, for the entropy measure, compute

$$H(T|x_1 = 1.5)$$

$$\text{Entropy} = -\sum_{i=1}^{k} p_i \cdot \log_2(p_i)$$

$$\text{Gini} = \sum_{i=1}^{k} p_i(1-p_i) = 1 - \sum_{i=1}^{k} p_i^2$$

What value of feature to split on? (4)



For each feature, do:

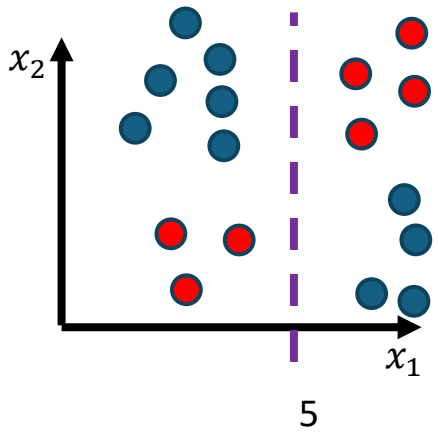
a. Sort the points from lowest to highest
b. Compute successive midpoints between each point in the dataset

c. Compute Entropy or Gini measures for all the midpoints

So, for the entropy measure, compute

$$H(T|x_1 = 2.2)$$

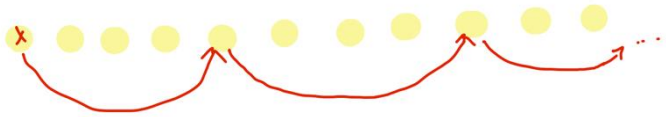
What value of feature to split on? (5)



Any problem?

usually, don't need to check all points.

"check every 10 points"



For each feature, do:

a. Sort the points from lowest to highest
b. Compute successive midpoints between each point in the dataset

c. Compute Entropy or Gini measures for all the midpoints

d. Compute information gain ΔG or ΔH for every possible split. Pick the one which has the highest Information gain

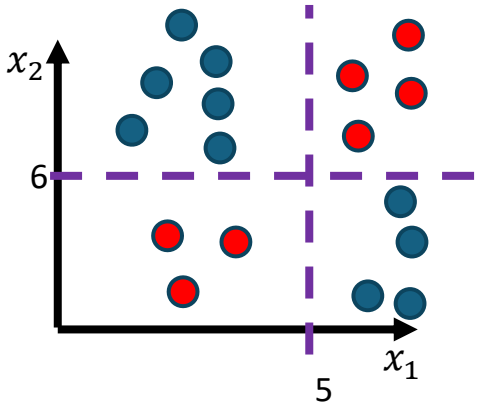
Decision Trees in action

Step-1: First compute the best cutpoint for *each feature* using Gini or Entropy information gain:

x_1=5 and x_2= 6

Step-2: Compute the best feature by comparing the Gini or Entropy information gain *across features:*

x_1=5 first, then x_2= 6

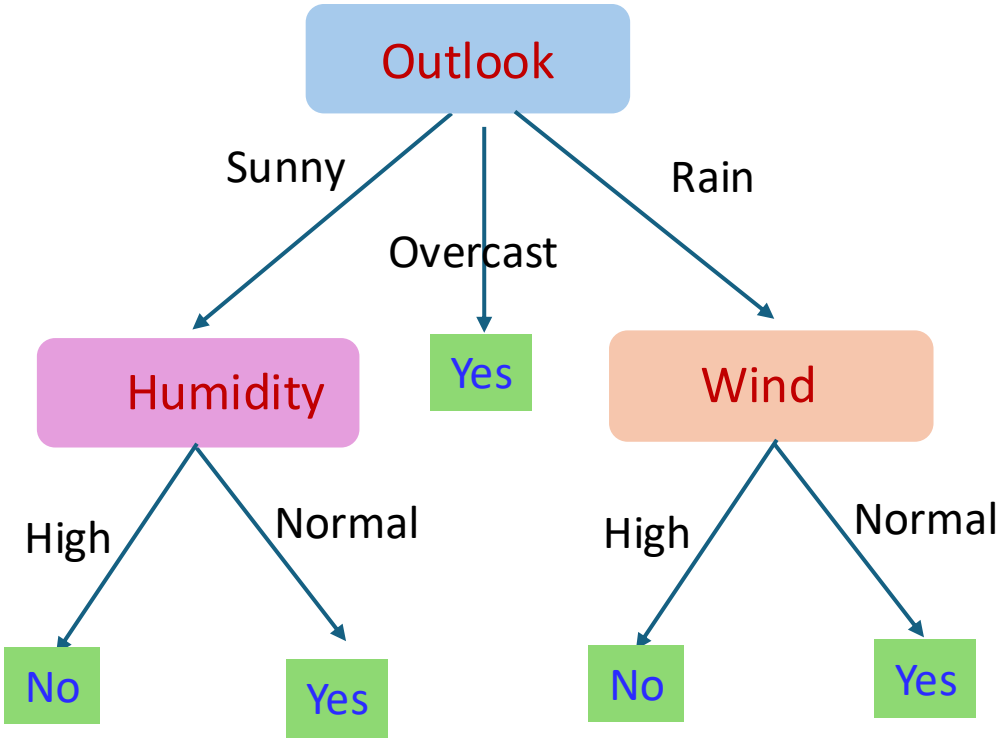


Outline (4)

- Decision Tree Classifier
- Feature Selection
- Build a Decision Tree Classifier
- Avoid Overfitting

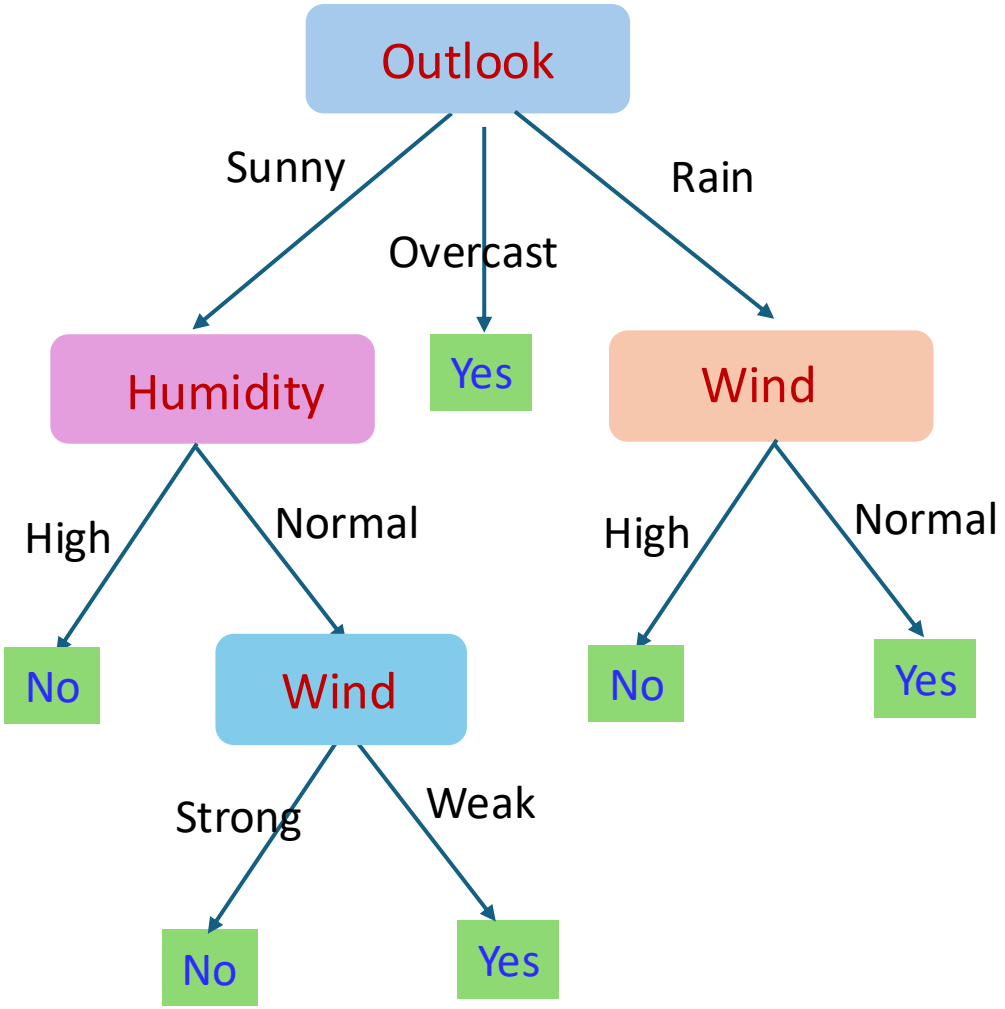
Overfitting -Example

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*



Overfitting –Example (Cont'd)

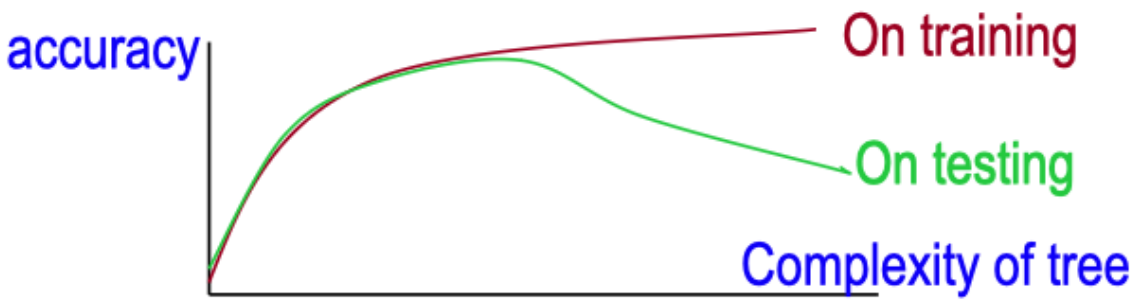
Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	*No*
Day2	Sunny	Hot	High	Strong	*No*
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	*No*
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	*No*
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	No
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	*No*



This can always be done – may fit noise or other coincidental regularities.

Overfitting the Data

- Learning a tree that classifies the training data perfectly may not lead to the tree with the best generalization performance.
  - There may be noise in the training data the tree is fitting
  - The algorithm might be making decisions based on very little data.



Common Factors leading to Overfitting in Decision Tree

- Decision Trees with too many splits
- Small training set
- Too many Features (High Dimensionality)
- No Pruning (Unrestricted growth)
- Bias in Data
  - Noise or outliers in Data

Prevent Overfitting in Decision Trees

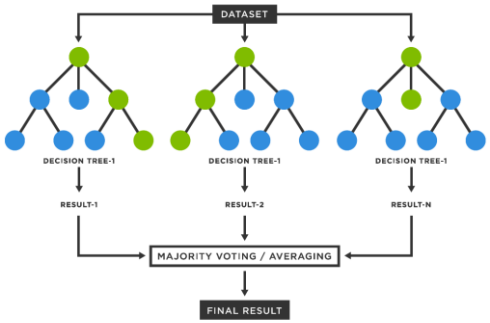
1. Pruning (Pre-pruning & Post-pruning)
   - Pre-pruning: Stop tree growth if the split does not improve information gain significantly.
   - Post-pruning: Grow a full tree first, then remove unnecessary branches using validation data.
2. Increasing Minimum Samples per Leaf
   - Restrict the *minimum number of samples required to split a node*.

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
```

3. Setting Maximum Depth
   - Set a *maximum depth* to prevent excessive growth.

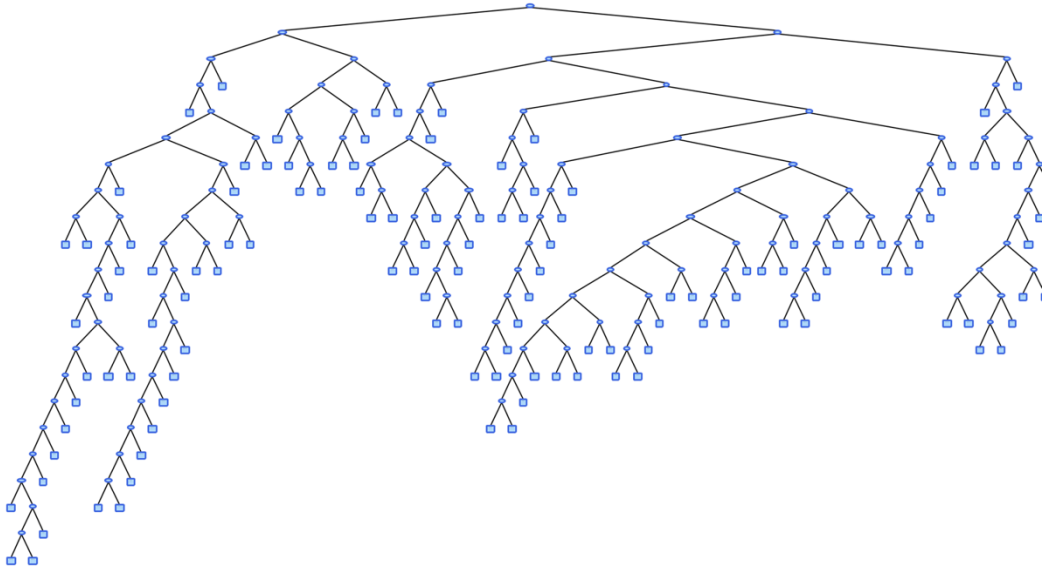
Prevent Overfitting in Decision Trees. (Cont'd)

4. Using Ensemble Methods (Random Forest, Boosting)
   - Instead of relying on a single decision tree, Random Forest builds multiple trees.
   - The final predication is made by averaging or majority voting, reducing variance and mitigating overfitting.



5. Feature Selection or Dimensionality Reduction
   - Help eliminate irrelevant or redundant features that may lead to unnecessary tree splits, reducing overfitting.
6. Cross-validation to Evaluate Generalization
   - CV helps assess how well a decision tree generalizes to new data, helping to detect overfitting before deployment.

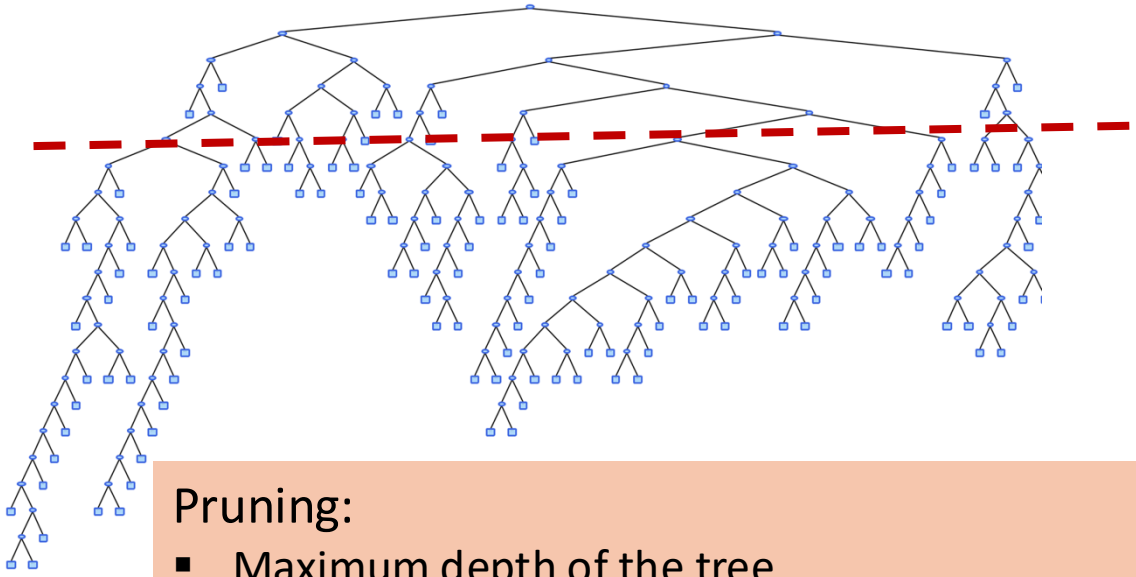
Overfitting - Example



Too deep a tree => Overfitting

Pruning: Pre-pruning

- Two basic approaches:
  - Pre-pruning: stop growing the tree at some point during construction (i.e., when it is determined that there is not enough data to make reliable choices) (early stopping).



Pruning:

- Maximum depth of the tree
- Minimum number of samples required to split an internal node
- Minimum number of samples required to be at a leaf node.

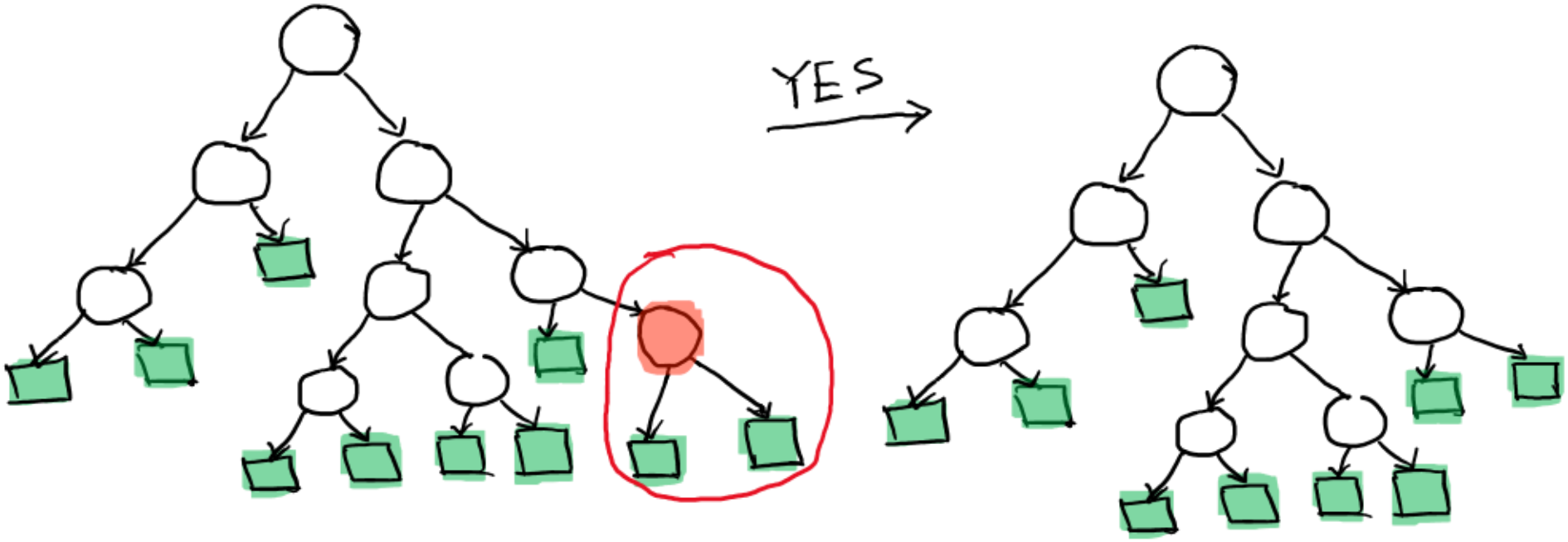
Pruning: Post-pruning

- Two basic approaches:
  - Pre-pruning: stop growing the tree at some point during construction (i.e., when it is determined that there is not enough data to make reliable choices).
  - Post-pruning: Grow the full tree and then remove nodes that seem not to have sufficient evidence. The idea is to remove branches that provide little predictive value, improving the generalization ability of the tree.

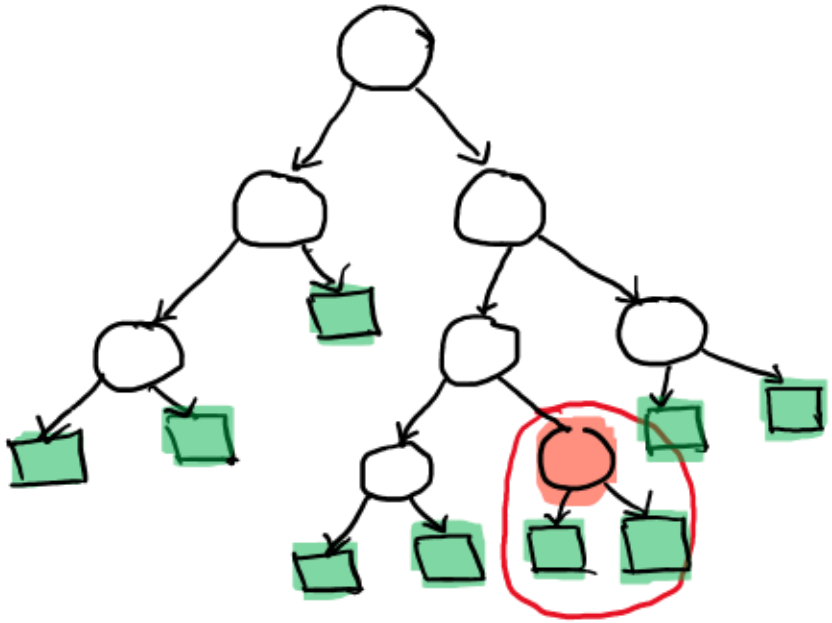
Steps of post-pruning:

1. Grow a fully-expanded decision tree
   The tree is allowed to grow until a stopping criterion is met. This typically results in an overfitted model.
2. Evaluate subtrees
   Subtrees are evaluated to see how much error they cause on a validation dataset.
3. Prune the weakest subtree
   Starting from the leaf nodes, decision trees are pruned by removing branches or subtrees that contribute minimally to model accuracy, i.e., compare the error rate of the tree before and after pruning.
4. Iterate
   This process is repeated until a subtree's removal does not significantly affect the performance on the validation dataset, and the tree becomes simpler, with reduced complexity.

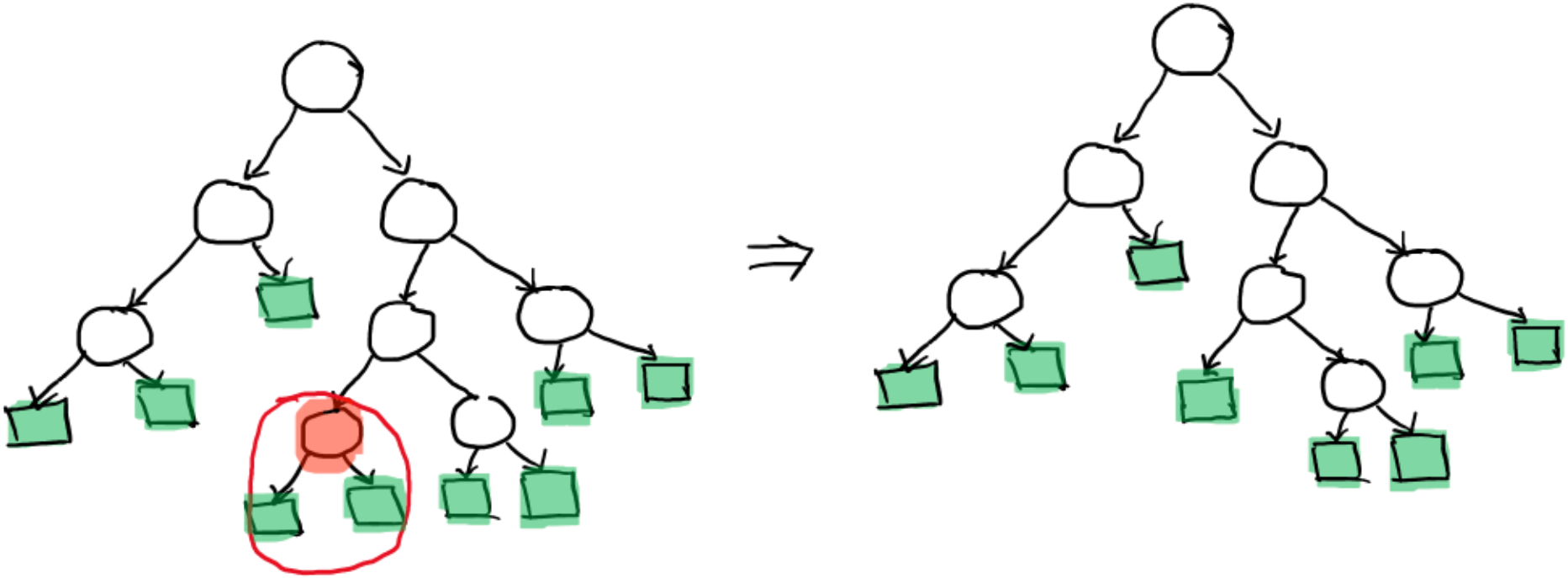
Post-pruning Example (1)



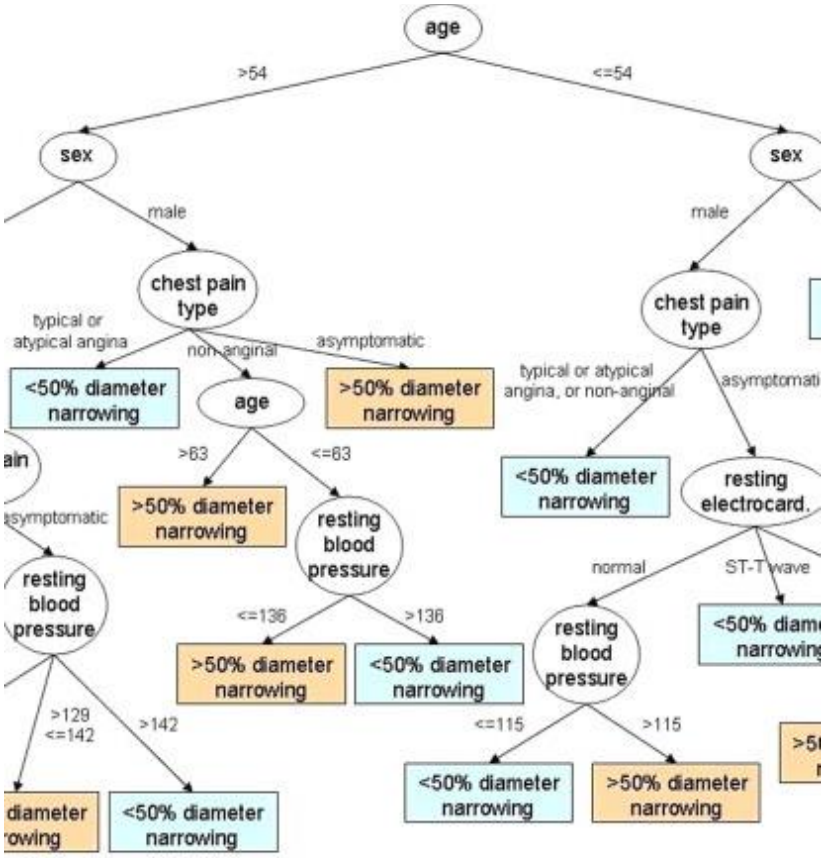
Post-pruning Example (2)



Post-pruning Example (3)



Interpretability: A Major Advantage



Decision trees are:

- **Human-readable**
- Explainable step-by-step
- Easy to debug

Example explanation:

- "This sample is classified as positive because feature A > 5 and feature B ≤ 3."

Why Interpretability Matters

Interpretability is critical in:

- Healthcare
- Finance
- Law
- Policy decisions

Regulations:

- “Right to explanation”
- Trust & accountability

Decision trees:

- Often preferred over black-box models

Limitations of Decision Trees

- Greedy → suboptimal splits
  - High variance
  - Unstable to small data changes
  - Axis-aligned splits only

Leads to:

- **Ensemble methods** (Random Forests, Boosting)

Summary

- Decision trees split data recursively
- Impurity measures:
  - Entropy → Information Gain
  - Gini index
- Strong interpretability
- Weak generalization alone

After-class Questions (Concepts)

- What does “node impurity” mean in a decision tree?
- Why do we need impurity measures to build a decision tree?
- Give an example of a feature that may be favored by Information Gain but is not useful for prediction.
- Why are decision trees prone to overfitting?
- Give two different ways to control the complexity of a decision tree.
- Why are decision trees considered interpretable models?
- Would you still choose a decision tree if a black-box model performs better? Why or why not?

After-class Questions (hands-on)

1. Computing Impurity by Hand;
2. One Split Decision (from scratch): given a dataset with multiple attributes, which attribute (and value) to split on at the root node?

After-class Questions (Further)

Exploring ID3, C4.5, CART algorithms.

And try to answer the following questions:

- Why does ID3 not support continuous features?
- How does C4.5 handle continuous features?
- Why does CART restrict splits to be binary?